

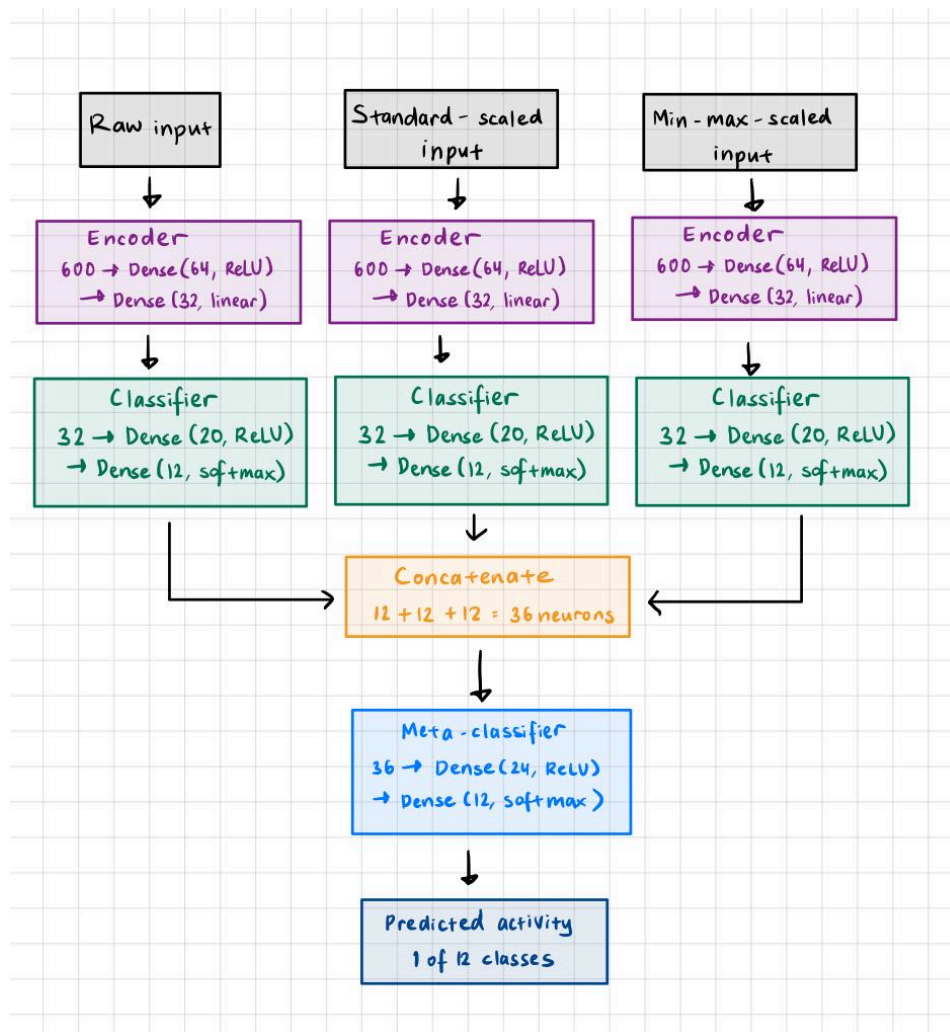
# Part I

Edge Impulse Project Link: <https://studio.edgeimpulse.com/public/1023599/live>

```
Last login: Tue Jun 9 02:21:47 on ttye002
tanisat@tanisat:~$ cd /home/tanisat/edge-impulse-run-impulse
Edge Impulse impulse runner v1.30.1
[SER] Connecting to /dev/tty.usbmodem101
[SER] Serial is connected, trying to read config...
Failed to parse snapshot line [ ]
[SER] Retrieved configuration
[SER] Device is running AT command version 1.8.0
[SER] Started inferencing, press CTRL-C to stop...
LSE
Inferencing settings:
  Interval: 20.00 ms.
  Frame size: 380
  Sample length: 2000 ms.
  No. of classes: 2
Starting inferencing, press 'b' to break
Starting inferencing in 2 seconds...
Sampling...
INFO: HW FFT failed, FFT size not supported. Must be a power of 2 between 32 and 4096, (size was 16)Timing: DSP 84 ms, inference 539 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  id1: 0.000000
  lift: 0.990000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 83 ms, inference 568 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  id1: 0.000000
  lift: 0.990000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 84 ms, inference 525 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  id1: 0.000000
  lift: 0.990000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 80 ms, inference 539 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  id1: 0.000000
  lift: 0.990000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 84 ms, inference 558 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  id1: 0.000000
  lift: 0.990000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 80 ms, inference 551 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  id1: 0.000000
  lift: 0.990000
Starting inferencing in 2 seconds...
#
```

## Part II

1.



### 2. Pruning and QAT methodology

- The notebook uses `prune_low_magnitude` with a `PolynomialDecay` schedule that starts at 10% sparsity, before going up to 80% over 500 steps and gradually zeroing out the smallest weights. An `UpdatePruningStep` callback drives this schedule forward each batch.
- Pruning wraps each layer with TF-MOT scaffolding. QAT also needs to wrap layers, and you can't wrap a wrapper. Hence, `strip_pruning` is called first instead to get a clean Keras model with sparse weights already in.
- `quantize_model` inserts fake quantization nodes that simulate 8-bit rounding during the forward pass. The model fine-tunes with these in place so that the weights adjust to quantization noise before actual conversion, which helps recover the accuracy that plain post-training quantization would otherwise lose.

- d. During QAT fine-tuning, gradient updates can push zeroed weights back to nonzero, which undoes the pruning. The MaskEnforcerCallback multiplies weights by a binary mask after every batch, forcing previously-zeroed weights to stay zero throughout QAT.
  - e. The converter needs real activation ranges to calculate the correct int8 scale and zero-point values per layer. The representative dataset is fed through so the converter can measure those ranges before finalizing the conversion.
  - f. The Arduino Nano has 256KB RAM and 1MB flash. 80% pruning means most weights are zero and compress well. Int8 quantization shrinks each weight from 4 bytes to 1 byte, cutting model size by around 5 times. Together, they make the model actually fit and run on hardware with no FPU.
- 3.

```
Tiny_Ensemble_Learning.ino  encoder_cif_minmax_pruned_qat_int8.cc  encoder_cif_raw_pruned_qat_int8.cc  encoder_cif_std_pruned_qat_int8.cc  stacked_meta_cif_pruned_qat_int8.cc
53  const float kgToMs2 = 9.80665f;

Output  Serial Monitor X
Message (Enter to send message to 'Arduino Nano 33 BLE' on '/dev/cu.usbmodem3101')
New Line 115200 baud

Final prediction: Jogging
Class index: 9
Confidence score: 0.4766

-----
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.2930, 0.0000, 0.1445, 0.1328, 0.0000, 0.0000, 0.0000, 0.0937, 0.0430, 0.2930
Standard-scaled model scores: 0.0000, 0.0000, 0.7227, 0.0000, 0.0039, 0.0000, 0.0391, 0.2344, 0.0000, 0.0000, 0.0000, 0.0000
Min-max-scaled model scores: 0.0000, 0.0039, 0.4570, 0.0000, 0.5078, 0.0039, 0.0117, 0.0000, 0.0000, 0.0000, 0.0000, 0.0117
Meta-classifier scores: 0.0117, 0.0391, 0.8516, 0.0195, 0.0195, 0.0000, 0.0000, 0.0312, 0.0000, 0.0156, 0.0078, 0.0039

Final prediction: Lying down
Class index: 2
Confidence score: 0.8516

-----
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0273, 0.0703, 0.3008, 0.0000, 0.1523, 0.1367, 0.0000, 0.0000, 0.0000, 0.0977, 0.0742, 0.1367
Standard-scaled model scores: 0.0000, 0.0039, 0.0508, 0.0000, 0.6523, 0.0000, 0.0039, 0.0000, 0.0000, 0.0000, 0.0000, 0.2969
Min-max-scaled model scores: 0.0000, 0.0039, 0.4375, 0.0000, 0.4883, 0.0000, 0.0586, 0.0000, 0.0000, 0.0000, 0.0000, 0.0078
Meta-classifier scores: 0.0156, 0.0703, 0.0625, 0.0039, 0.6094, 0.0039, 0.0000, 0.2031, 0.0078, 0.0078, 0.0078, 0.0039

Final prediction: Climbing stairs
Class index: 4
Confidence score: 0.6094

-----
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0391, 0.9570, 0.0000
Standard-scaled model scores: 0.0000, 0.0000, 0.0000, 0.0000, 0.9766, 0.0000, 0.0000, 0.0000, 0.0000, 0.0117, 0.0000, 0.0117
Min-max-scaled model scores: 0.0000, 0.0000, 0.3555, 0.0000, 0.3984, 0.0078, 0.2266, 0.0000, 0.0000, 0.0000, 0.0078, 0.0000
Meta-classifier scores: 0.0469, 0.0547, 0.0469, 0.0039, 0.5039, 0.0352, 0.0312, 0.1758, 0.0352, 0.0
```

When the board is resting flat, the model consistently predicts the same class, which is acceptable behavior for a stationary board. When I moved or rotated it, however, the predictions got more unstable and jumped between classes. This is due to the fact that the model was trained on the mHealth dataset from sensors worn on a human body, so random manual rotations of a bare Arduino don't really match any training class cleanly. Two things that could help are either adding a confidence threshold so low-confidence predictions just output "uncertain" instead of forcing a label, or recalibrating the scaling parameters to better match what the Nano's IMU actually outputs.

4. Yes, the model did predict the same class repeatedly even when I moved the board around. The biggest reason for this might be the dataset mismatch; mHealth data was

collected from sensors strapped to a person's body doing structured activities, which is pretty different from a bare Arduino being moved around on a desk (or even placed on my chest like I tried). The axis orientation is also likely different between the mHealth sensor and the Nano's IMU, so the raw values the board sees don't line up with what the model learned. On top of that, even though the sketch converts unit, there could still be subtle scaling differences between the training data and what the board actually outputs that push predictions toward the same class repeatedly.